```
$userData = json_decode($userInput, true);

var_dump($userData);

if ($secretHexValue == $userData['token']) {
    print "I just got hacked :(" . PHP_EOL;
}

Output: (PHP 7.4)
array(1) {
  ["token"]=>
  int(0)
}
I just got hacked :(
```

In the code above, the built-in `json_decode()` function is used to transform the `$userData` JSON string to an associative array. JSON supports a limited set of data types, and in this case `"token": 0` means the "token" property has an integer value *0* (which is confirmed in the `var_dump()` output).

Our hypothetical hacker is passing a malicious JSON payload, and the loosely typed comparison between `$secretHexValue` and `$userData['token']` results in the secret string being cast to an integer. It's true that this vulnerability will only trigger under certain conditions (the secret hash string needs to have either leading '0' digits followed by a letter or the letters A-F as the first character, and it will cast to *0*), so it's only a vulnerability some of the time. But hackers are persistent, and eventually they'll get you…

This demonstration might look like a lot of the rest of the code in this book (contrived to demonstrate a single point), but it is actually a simplified version of a real-world exploit that existed in the Laravel framework some years ago (one that compromised cross-site request forgery protection).

The same code running under PHP 8 would not trigger the issue: the rules around type coercion and loosely typed comparison are different from previous versions (we'll take a closer look in a moment). Another fix is, of course, to use a strongly typed comparison between `$secretHexValue` and the malicious token value.

```
if ($secretHexValue === $userData['token']) {
```